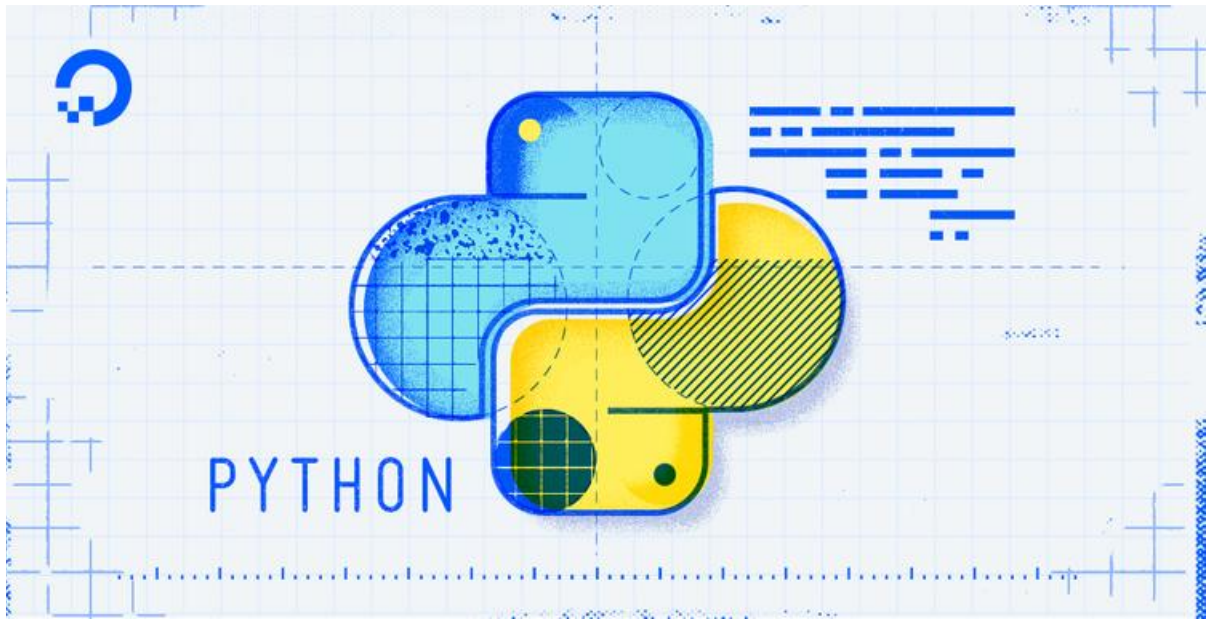




Python

Collections, functions and Modules

-ved jani

Accessing List

Q.Understanding how to create and access elements in a list.

Ans:

1. Creating a List

- A list in Python is made using square brackets [].
- It can store multiple values, even of different data types.
- Example:
 - `numbers = [10, 20, 30, 40]`
 - `names = ["ved", "raj", "amit"]`
 - `mixed = [10, "hello", 3.14]`

2. Accessing Elements in a List

- Each item has an index number.
- Index starts from 0 for the first element.
- Example:
 - `numbers[0]` # gives 10
 - `numbers[2]` # gives 30

3. Negative Indexing

- Python also allows negative indexes.
- -1 refers to the last element.
- Example:
 - `numbers[-1]` # gives 40
 - `numbers[-2]` # gives 30

4. Using Slicing to Access Multiple Elements

- Slicing gives a range of values.
- Format: `list[start:end]`
(end is not included)
- Example:
 - `numbers[1:3]` # gives [20, 30]
 - `numbers[:2]` # gives [10, 20]
 - `numbers[2:]` # gives [30, 40]

5. Changing List Elements

- Lists are mutable, meaning we can change values.
- Example:

numbers[1] = 25 # now list becomes [10, 25, 30, 40]

Q. Indexing in lists (positive and negative indexing).

Ans:

Indexing in Lists

1. Positive Indexing

- In Python, list indexing starts from 0.
- The first element is at index 0, second at 1, and so on.

Example:

- fruits = ["apple", "banana", "mango", "orange"]
- fruits[0] # apple
- fruits[2] # mango

2. Negative Indexing

- Negative indexing starts from **-1** for the **last element**.
- -1 means last item, -2 means second last, and so on.

Example:

- fruits[-1] # orange
- fruits[-3] # banana

Short Summary

- **Positive indexing:** left to right (0, 1, 2, ...)
- **Negative indexing:** right to left (-1, -2, -3, ...)

Q. Slicing a list: accessing a range of elements.

Ans:

Slicing a List (Accessing a Range of Elements)

1. What is Slicing?

- Slicing means taking a portion (subset) of a list.
- It is done using the format:
list[start : end]

•

2. How It Works

- **start** → index where the slice begins
- **end** → index where the slice stops (but end index is not included)

3. Example

```
numbers = [10, 20, 30, 40, 50]  
numbers[1:4] # gives [20, 30, 40]
```

4. Omitting Start or End

- If you skip start, Python starts from index 0.
- numbers[:3] # [10, 20, 30]
- If you skip end, Python goes to the last element.
- numbers[2:] # [30, 40, 50]

5. Slicing With Step

- You can also add a step value:
list[start:end:step]
- Example:
- numbers[0:5:2] # [10, 30, 50]

Short Summary

- Slicing returns multiple elements.
- End index is excluded.
- Format: list[start:end:step].

List Operations

Q. Common list operations: concatenation, repetition, membership.

Ans:

1. Concatenation (Joining Lists)

- Used to combine two or more lists.
- Operator: `+`
- Example:
 - `list1 = [1, 2]`
 - `list2 = [3, 4]`
 - `result = list1 + list2` # `[1, 2, 3, 4]`

2. Repetition (Repeating Elements)

- Used to repeat the list multiple times.
- Operator: `*`
- Example:
 - `nums = [5, 6]`
 - `result = nums * 3` # `[5, 6, 5, 6, 5, 6]`

3. Membership (Checking if an element exists)

- Used to check whether an item is present in a list.
- **Operators:** `in` and `not in`
- Example:
 - `fruits = ["apple", "banana", "mango"]`
 - `"banana" in fruits` # `True`
 - `"orange" not in fruits` # `True`

Short Summary

- **Concatenation:** joins lists → `list1 + list2`
- **Repetition:** repeats list → `list * n`
- **Membership:** checks presence → `"item" in list`

Q. Understanding list methods like append(), insert(), remove(), pop().

Ans:

1. append()

- Adds an element at the end of the list.
- Example:
- `fruits = ["apple", "banana"]`
- `fruits.append("mango")`
- `# ["apple", "banana", "mango"]`

2. insert()

- Adds an element at a specific index.
- Format: `list.insert(index, value)`
- Example:
- `fruits = ["apple", "banana"]`
- `fruits.insert(1, "grapes")`
- `# ["apple", "grapes", "banana"]`

3. remove()

- Removes the first occurrence of a value.
- If the value does not exist, it gives an error.
- Example:
- `fruits = ["apple", "banana", "apple"]`
- `fruits.remove("apple")`
- `# removes the first "apple"`
- `# ["banana", "apple"]`

4. pop()

- Removes an element from a specific index.
- If no index is given, it removes the last element.
- Example:
- `fruits = ["apple", "banana", "mango"]`
- `fruits.pop(1) # removes "banana"`

`fruits.pop() # removes last element`

Working with Lists

Q. Iterating over a list using loops.

Ans:

Iterating Over a List Using Loops

1. Using a for loop

- The most common way to go through each element in a list.

Example:

```
fruits = ["apple", "banana", "mango"]
```

```
for item in fruits:
```

```
    print(item)
```

Output:

```
    apple
```

```
    banana
```

```
    mango
```

2. Using a for loop with index (range + len)

- Used when you need the index number.

Example:

```
numbers = [10, 20, 30]
```

```
for i in range(len(numbers)):
```

```
    print(i, numbers[i])
```

Output:

```
0 10
```

```
1 20
```

```
2 30
```

3. Using a while loop

- Useful when you want to manually control the index.

Example:

```
i = 0
```

```
items = ["a", "b", "c"]
```

```
while i < len(items):
```

```
    print(items[i])    i += 1
```

Q. Sorting and reversing a list using sort(), sorted(), and reverse().

Ans:

Sorting and Reversing a List in Python

1. sort()

- sort() changes the original list (in-place sorting).
- By default, it sorts the list in ascending order.
- **Example:**

```
numbers = [3, 1, 4, 2]
numbers.sort()
# Result: [1, 2, 3, 4]
```

- **Sorting in descending order:**

```
numbers.sort(reverse=True)
# Result: [4, 3, 2, 1]
```

2. sorted()

- sorted() returns a new sorted list.
- It does not modify the original list.
- **Example:**

```
nums = [5, 2, 8]
new_list = sorted(nums)
# new_list: [2, 5, 8]
# nums remains: [5, 2, 8]
```

3. reverse()

- reverse() simply reverses the order of the list.
- It does not sort; it only flips the elements.
- **Example:**

```
items = [10, 20, 30]
items.reverse()
# Result: [30, 20, 10]
```


Q. Basic list manipulations: addition, deletion, updating, and slicing.

Ans:

Basic List Manipulations in Python

1. Addition (Adding Elements)

- Elements can be added using:
 - **append()** → adds an item at the end
 - **insert()** → adds an item at a specific index
 - **extend()** → adds multiple elements
- **Example:**

```
numbers = [10, 20]
numbers.append(30)    # [10, 20, 30]
numbers.insert(1, 15) # [10, 15, 20, 30]
numbers.extend([40, 50]) # [10, 15, 20, 30, 40, 50]
```

2. Deletion (Removing Elements)

- Elements can be removed using:
 - **remove()** → removes by value
 - **pop()** → removes by index (default: last item)
 - **del** → deletes by index or the whole list
- **Example:**

```
numbers = [10, 20, 30, 40]
numbers.remove(20) # [10, 30, 40]
numbers.pop(1)    # removes 30 → [10, 40]
del numbers[0]    # deletes 10 → [40]
```

3. Updating (Changing Elements)

- Lists are mutable, so you can change values directly.
- **Example:**

```
items = ["apple", "banana", "mango"]
items[1] = "orange"    # ["apple", "orange", "mango"]
```

4. Slicing (Accessing a Range of Elements)

- Slicing returns a portion of the list.
- Format: list[start : end] (end index not included)

- **Example:**

```
numbers = [10, 20, 30, 40, 50]
```

```
numbers[1:4] # [20, 30, 40]
```

```
numbers[:3] # [10, 20, 30]
```

```
numbers[2:] # [30, 40, 50]
```

```
numbers[0:5:2] # [10, 30, 50] (with step)
```

Tuple

Q. Introduction to tuples, immutability

Ans:

Introduction to Tuples and Immutability

1. What is a Tuple?

- A tuple is an ordered collection of elements in Python.
- It is written using round brackets ().
- A tuple can store different types of data just like a list.
- **Example:**

```
my_tuple = (10, 20, 30)
mixed_tuple = (10, "apple", 3.5)
```

2. Immutability of Tuples

- The most important feature of a tuple is immutability.
- Immutability means the elements of a tuple cannot be changed after it is created.
- You cannot add, delete, or update elements in a tuple.

Example:

```
numbers = (1, 2, 3)
numbers[1] = 5 # Not allowed (will give an error)
```

3. Why Use Tuples?

- They are faster than lists.
- Useful when you want to store fixed data that should not change.
- Can be used as keys in dictionaries (because they are immutable).

4. Accessing Tuple Elements

- Tuple indexing works the same as lists.
- ```
t = ("apple", "banana", "mango")
t[0] # "apple"
t[-1] # "mango"
```

## Q. Creating and accessing elements in a tuple.

Ans:

### Creating and Accessing Elements in a Tuple

#### 1. Creating a Tuple

- A tuple is created using round brackets ().
- It can contain numbers, strings, or mixed data types.

- **Examples:**

```
t1 = (10, 20, 30)
```

```
t2 = ("apple", "banana", "mango")
```

```
t3 = (10, "hello", 3.5)
```

Creating a single-element tuple

- **You must add a comma after the value.**

```
t = (5,) # single-element tuple
```

#### 2. Accessing Elements in a Tuple

- Tuple elements are accessed using index numbers.
- Indexing starts from 0 (first element).
- Negative indexing is also allowed.

#### Examples:

```
fruits = ("apple", "banana", "mango", "orange")
```

```
fruits[0] # "apple"
```

```
fruits[2] # "mango"
```

```
fruits[-1] # "orange" (last element)
```

## Q. Basic operations with tuples: concatenation, repetition, membership.

Ans:

### Basic Operations with Tuples in Python

#### 1. Concatenation

- Concatenation means joining two tuples together.
- Operator used: +
- **Example:**

```
t1 = (1, 2)
t2 = (3, 4)
result = t1 + t2
Result: (1, 2, 3, 4)
```

#### 2. Repetition

- Repetition means repeating a tuple multiple times.
- Operator used: \*
- **Example:**

```
t = (5, 6)
result = t * 3
Result: (5, 6, 5, 6, 5, 6)
```

#### 3. Membership

- Checks whether a value exists in the tuple or not.
- Operators: in and not in
- **Example:**

```
fruits = ("apple", "banana", "mango")
```

```
"banana" in fruits # True
```

```
"orange" not in fruits # True
```

# Accessing Tuples

## Q. Accessing tuple elements using positive and negative indexing.

Ans:

### Accessing Tuple Elements Using Positive and Negative Indexing

#### 1. Positive Indexing

- Positive indexing starts from 0 for the first element.
- It moves from left to right.

- **Example:**

```
fruits = ("apple", "banana", "mango", "orange")
```

```
fruits[0] # "apple"
```

```
fruits[2] # "mango"
```

#### 2. Negative Indexing

- Negative indexing starts from -1 for the last element.
- It moves from right to left.

- **Example:**

```
fruits = ("apple", "banana", "mango", "orange")
```

```
fruits[-1] # "orange" (last element)
```

```
fruits[-3] # "banana"
```

## Q. Slicing a tuple to access ranges of elements.

Ans:

### Slicing a Tuple to Access Ranges of Elements

#### 1. What is Slicing?

- Slicing means extracting a portion (range) of elements from a tuple.
- **The format is:**

`tuple[start : end]`

- The end index is not included in the result.

#### 2. Basic Slicing Example

`numbers = (10, 20, 30, 40, 50)`

`numbers[1:4]` # Result: (20, 30, 40)

#### 3. Slicing Without Start or End

- If start is missing: slicing begins from index 0.

`numbers[:3]` # (10, 20, 30)

- If end is missing: slicing goes up to the last element.

`numbers[2:]` # (30, 40, 50)

#### 4. Slicing With Step Value

- **You can add a step to skip elements:**

`numbers[0:5:2]` # (10, 30, 50)

# Dictionaries

## Q. Introduction to dictionaries: key-value pairs.

Ans:

### Introduction to Dictionaries: Key-Value Pairs

#### 1. What is a Dictionary?

- A dictionary in Python is a collection of data stored in key-value pairs.
- It is written using curly braces {}.
- Each value is accessed using its key, not an index (like lists/tuples).
- **Example:**

```
student = {
 "name": "Ved",
 "age": 20,
 "course": "Python"
}
```

#### 2. Key-Value Pairs

- **A dictionary stores data in the format:**

**key : value**

- Key acts like a label (must be unique).
- Value is the data stored under that key.

**Example:**

"age" : 20

#### 3. Features of Dictionaries

- Keys must be unique.
- Values can be any data type (string, number, list, etc.).
- Dictionaries are unordered collections (order not guaranteed in older Python versions).
- Dictionaries are mutable (values can be changed).



#### 4. Accessing Values

- Values are accessed by using keys, not indexes.

```
student["name"] # "Ved"
```

```
student["age"] # 20
```

### Q. Accessing, adding, updating, and deleting dictionary elements.

Ans:

#### Accessing, Adding, Updating, and Deleting Dictionary Elements

##### 1. Accessing Dictionary Elements

- You can access values using their keys.

- **Example:**

```
student = {"name": "Ved", "age": 20, "course": "Python"}
```

```
student["name"] # "Ved"
```

```
student["age"] # 20
```

- **Use .get() to safely access values:**

```
student.get("course") # "Python"
```

##### 2. Adding Elements to a Dictionary

- Add a new key–value pair by assigning a value to a new key.

```
student["city"] = "Ahmedabad"
```

```
{"name": "Ved", "age": 20, "course": "Python", "city": "Ahmedabad"}
```

##### 3. Updating Dictionary Elements

- Change the value of an existing key.

```
student["age"] = 21
```

```
student["course"] = "Full Stack"
```

- **You can also update multiple values using .update():**

```
student.update({"age": 22, "city": "Surat"})
```

#### 4. Deleting Elements from a Dictionary

- `del` → removes a specific key.  
`del student["city"]`
- `pop()` → removes a key and returns its value.  
`student.pop("age")`
- `clear()` → removes all items from the dictionary.  
`student.clear()` # {}

#### Q. Dictionary methods like `keys()`, `values()`, and `items()`.

Ans:

#### Dictionary Methods: `keys()`, `values()`, and `items()`

##### 1. `keys()` Method

- Returns a list-like view of all keys in the dictionary.
- **Example:**  
`student = {"name": "Ved", "age": 20, "course": "Python"}`  
  
`student.keys()`  
# Output: dict\_keys(['name', 'age', 'course'])

##### 2. `values()` Method

- Returns a list-like view of all values in the dictionary.
- **Example:**  
`student.values()`  
# Output: dict\_values(['Ved', 20, 'Python'])

##### 3. `items()` Method

- Returns all key–value pairs as tuples.
- **Example:**  
`student.items()`  
# Output: dict\_items([  
# ('name', 'Ved'),  
# ('age', 20),  
# ('course', 'Python')  
# ])

# Working with Dictionaries

## Q. Iterating over a dictionary using loops.

Ans:

### Iterating Over a Dictionary Using Loops

#### 1. Iterating Over Keys (Default Behavior)

- When you loop over a dictionary, it gives keys by default.

```
student = {"name": "Ved", "age": 20, "course": "Python"}
```

for key in student:

```
 print(key)
```

**Output:**

name

age

course

#### 2. Iterating Over Values

- Use the .values() method to access only the values.

```
for value in student.values():
```

```
 print(value)
```

**Output:**

Ved

20

Python

#### 3. Iterating Over Key-Value Pairs

- Use the .items() method to access both key and value at the same time.

```
for key, value in student.items():
```

```
 print(key, ":", value)
```

**Output:**

name : Ved

age : 20

course : Python

## Q. Merging two lists into a dictionary using loops or zip().

**Ans:**

### Merging Two Lists into a Dictionary Using Loops or zip()

#### 1. Using a Loop

- You can merge two lists by pairing each element using a loop.
- The first list becomes keys, and the second becomes values.

**Example:**

```
keys = ["name", "age", "city"]
```

```
values = ["Ved", 20, "Ahmedabad"]
```

```
result = {}
```

```
for i in range(len(keys)):
```

```
 result[keys[i]] = values[i]
```

```
Result: {'name': 'Ved', 'age': 20, 'city': 'Ahmedabad'}
```

#### 2. Using zip()

- zip() pairs elements from both lists automatically.
- This is the simplest and most common way.

**Example:**

```
keys = ["name", "age", "city"]
```

```
values = ["Ved", 20, "Ahmedabad"]
```

```
result = dict(zip(keys, values))
```

```
Result: {'name': 'Ved', 'age': 20, 'city': 'Ahmedabad'}
```

## **Q. Counting occurrences of characters in a string using dictionaries.**

**Ans:**

### **Counting Occurrences of Characters in a String Using Dictionaries**

#### **1. Concept**

- A dictionary can be used to count how many times each character appears in a string.
- Each character becomes a key, and its count becomes the value.

#### **2. Example Program**

```
text = "hello"
```

```
count = {}
```

```
for char in text:
```

```
 if char in count:
```

```
 count[char] += 1
```

```
 else:
```

```
 count[char] = 1
```

```
print(count)
```

#### **3. Output**

```
{'h': 1, 'e': 1, 'l': 2, 'o': 1}
```

#### **4. How It Works**

- Loop through each character in the string.
- If the character already exists in the dictionary, increase its count.
- If not, add it to the dictionary with count 1.

# Functions

## Q. Defining functions in Python.

Ans:

### Defining Functions in Python

#### 1. What is a Function?

- A function is a block of reusable code that performs a specific task.
- It helps reduce repetition and makes programs easier to manage.

#### 2. Defining a Function

- In Python, a function is defined using the def keyword.
- **Basic structure:**

```
def function_name():
 # body of the function
 # statements
```

#### 3. Example of a Simple Function

**def greet():**

```
print("Hello, welcome to Python!")
```

- **To call the function:**

```
greet()
```

#### 4. Function With Parameters

- Parameters allow the function to receive input values.

**Example:**

```
def add(a, b):
 return a + b
```

```
result = add(5, 3) # result = 8
```

## 5. Return Statement

- return sends a value back from the function.
- A function can return numbers, strings, lists, or any data.

### Example:

```
def square(n):
 return n * n
```

## Q. Different types of functions: with/without parameters, with/without return values.

Ans:

### Different Types of Functions in Python

Functions in Python can be classified based on parameters and return values.

#### 1. Functions Without Parameters

- These functions do not take any input.
- **Example:**

```
def greet():
 print("Hello!")
```

- **Call:**

```
greet()
```

#### 2. Functions With Parameters

- These functions take input values.
- Parameters are used to pass data into the function.
- **Example:**

```
def add(a, b):
 print(a + b)
```

- **Call:**

```
add(5, 3) # Output: 8
```

### 3. Functions Without Return Values

- These functions do not return anything; they only perform an action (like printing).

- **Example:**

```
def show():
 print("This function has no return value.")
```

### 4. Functions With Return Values

- These functions send back a value using the return statement.

- **Example:**

```
def multiply(a, b):
 return a * b
```

```
result = multiply(4, 5) # result = 20
```

### Q. Anonymous functions (lambda functions).

**Ans:**

#### Anonymous Functions (Lambda Functions)

##### 1. What is an Anonymous Function?

- An anonymous function is a function without a name.
- In Python, anonymous functions are created using the lambda keyword.
- They are also called lambda functions.

##### 2. Syntax of a Lambda Function

lambda arguments: expression

- It can have any number of arguments, but only one expression.
- The expression is automatically returned.

##### 3. Simple Example

```
square = lambda x: x * x
print(square(5))
```

**Output:**25



#### 4. Lambda Function With Multiple Arguments

```
add = lambda a, b: a + b
print(add(3, 4))
```

**Output:**7

#### 5. Where Are Lambda Functions Used?

- Useful for short, simple operations.
- **Commonly used inside:**
  - map()
  - filter()
  - sorted()
  - inline calculations

##### **Example with map():**

```
numbers = [1, 2, 3]
result = list(map(lambda x: x * 2, numbers))
[2, 4, 6]
```

# Modules

## Q. Introduction to Python modules and importing modules.

**Ans:**

Introduction to Python Modules and Importing Modules

Modules in Python are files that contain reusable code such as functions, variables, and classes.

They help make programs more organized and allow you to use pre-written Python features.

### 1. What is a Module?

- A module is a Python file (.py) containing code that can be reused.
- It can include functions, variables, and classes.
- Python has many built-in modules like math, random, datetime.

**Example:**

```
This is a module file: myModule.py
```

```
def greet():
 print("Hello from module!")
```

### 2. Importing a Module

- To use a module in your program, you import it using the import keyword.

**Example:**

```
import math
```

```
print(math.sqrt(25))
```

Call:

```
math.sqrt(25) # Output: 5.0
```

### 3. Importing Specific Functions

- You can import only selected functions using from.

**Example:**

```
from math import sqrt, pi
```

```
print(sqrt(16))
```

```
print(pi)
```

**Call:**

`sqrt(16), pi`

**4. Importing a Module With an Alias**

- Alias helps shorten the module name.

**Example:**

```
import math as m
```

```
print(m.sqrt(9))
```

**Call:**

```
m.sqrt(9)
```

**5. Importing Your Own Module**

- You can create your own module and import it.

**Example:**

```
myModule.py
```

```
def hello():
```

```
 print("Hello Ved!")
```

**main file:**

```
import myModule
```

```
myModule.hello()
```

**Call:**

```
myModule.hello()
```

## **Q. Standard library modules: math, random.**

**Ans:**

### **Standard Library Modules: math and random**

#### **1. math Module**

- The math module provides mathematical functions and constants.

**Example:**

```
import math
```

```
print(math.sqrt(25))
print(math.pow(2, 3))
print(math.pi)
```

**Call:**

```
math.sqrt(25)
math.pow(2, 3)
math.pi
```

#### **2. random Module**

- The random module is used to generate random numbers and selections.

**Example:**

```
import random
```

```
print(random.randint(1, 10))
print(random.random())
print(random.choice(["apple", "banana", "mango"]))
```

**Call:**

```
random.randint(1, 10)
random.random()
random.choice(...)
```

## **Q. Creating custom modules.**

**Ans:**

Creating Custom Modules in Python

Custom modules are Python files created by the user to store reusable functions, variables, or classes.

### **1. What is a Custom Module?**

- A custom module is simply a Python file (.py) created by you.
- It allows code reuse across multiple programs.

### **2. Creating Your Own Module**

- **Step 1:** Create a file with a name like myModule.py.
- **Step 2:** Write functions inside it.

**Example (myModule.py):**

```
def greet(name):
 return "Hello " + name
```

```
def add(a, b):
 return a + b
```

### **3. Importing Your Custom Module**

- Use the import keyword to use your module in another program.

**Example (main program):**

```
import myModule

print(myModule.greet("Ved"))
print(myModule.add(5, 3))
```

### **4. Importing Specific Functions**

- You can import only selected functions.

**Example:**

```
from myModule import greet

print(greet("Rahul"))
```

**Call:**

```
myModule.greet("Ved")
```

```
myModule.add(5, 3)
```